Development and Validation of mc_auc_ci()

Stratified Bootstrap Confidence Intervals for Multiclass AUC

Thanh Hien Tran

Purpose

My task is to develop `mc_auc_ci()`. The main purpose of this function is to estimate the uncertainty around a Hand-Till multiclass AUC estimate

For example, if a model has a multiclass AUC of 0.82, this gives us one estimate of the model performance from the observed sample. However, the value 0.82 by itself does not show how much the estimate might change if the sample was slightly different

My current idea is to use bootstrap resampling to investigate this uncertainty. The function repeatedly resamples the observations, recalculates the multiclass AUC, and uses the resulting bootstrap AUC values to estimate the variability around the original AUC

Based on the supervisor's feedback, I am using stratified bootstrap resampling. Instead of sampling from the whole dataset at once, I resample observations separately within each class. This makes sure that every class remains represented in each bootstrap sample

For the first prototype, I want `mc_auc_ci()` to return:

- the original multiclass AUC estimate
- a confidence interval
- the bootstrap standard error
- the number of successful and failed bootstrap replicates
- the bootstrap AUC values for later visualisation

At this stage, I mainly want to check whether the bootstrap procedure and function structure are going in the correct direction. I am using a percentile confidence interval as the first implementation, but I have not treated this as the final CI method yet. The next step will be to test its coverage using simulation.

Note: The problem I'm trying to solve is that the multiclass AUC gives us one point estimate but it doesn't tell us much about the uncertainty around that estimate. So my function repeatedly resamples the data and recalculates the AUC to see how much it changes

What my function adds

`pROC` already provides the underlying Hand-Till multiclass AUC calculation, so I do not need to recreate this calculation

What I am adding is the uncertainty estimation around the multiclass AUC. My function takes the AUC calculation and places it inside a stratified bootstrap procedure

For each bootstrap replicate, the function:

1. resamples observations within each class
2. creates a new bootstrap sample
3. recalculates the multiclass AUC using `pROC`
4. stores the resulting AUC value

After repeating this many times, I can use the stored AUC values to estimate the bootstrap standard error and a confidence interval.

So the new part is not another AUC formula. The main part I am developing is the bootstrap procedure around the existing multiclass AUC calculation

Note: pROC is doing the underlying Hand-Till AUC calculation. My function adds the stratified resampling procedure around it, repeats the AUC calculation, handles failed replicates and then summarises the bootstrap distribution using SE and a confidence interval

Small Development Example

I first use a small example with three classes so that I can check each step of the bootstrap procedure before applying it to larger data. This example is only for development and is not the final validation dataset.

```
#example data
truth <- factor(c(
  "A", "A", "A", "A",
  "B", "B", "B", "B",
  "C", "C", "C", "C"))

table(truth)
```

```
## truth
## A B C
## 4 4 4
```

Each observation also needs predicted probabilities for the three classes. The rows of the probability matrix correspond to the observations in `truth`, and the columns correspond to classes A, B and C

Predicted probabilities

For this small example, I intentionally do not make the predicted probabilities perfectly separated between the three classes.

Some observations are easy to classify, while some are more ambiguous. I did this because if every observation is perfectly separated, the AUC can remain equal to 1 across the bootstrap samples

In that situation, the bootstrap standard error becomes 0 and the example does not really help me check whether the uncertainty part of the function is working

For example, one observation from class A has probabilities:

- A = 0.35
- B = 0.50

- C = 0.15

This is intentionally a difficult observation because the highest predicted probability is B even though the true class is A

This example is only used to develop and inspect the function. It is not my final validation dataset.

```
#example data
probabilities <- matrix(c(
  # True class A
  0.70, 0.20, 0.10,
  0.55, 0.30, 0.15,
  0.45, 0.40, 0.15,
  0.35, 0.50, 0.15,

  # True class B
  0.20, 0.65, 0.15,
  0.30, 0.50, 0.20,
  0.40, 0.45, 0.15,
  0.15, 0.40, 0.45,

  # True class C
  0.10, 0.20, 0.70,
  0.20, 0.25, 0.55,
  0.25, 0.40, 0.35,
  0.35, 0.20, 0.45
), ncol = 3, byrow = TRUE)
colnames(probabilities) <- c("A", "B", "C")
probabilities
```

```
##          A    B    C
##  [1,] 0.70 0.20 0.10
##  [2,] 0.55 0.30 0.15
##  [3,] 0.45 0.40 0.15
##  [4,] 0.35 0.50 0.15
##  [5,] 0.20 0.65 0.15
##  [6,] 0.30 0.50 0.20
##  [7,] 0.40 0.45 0.15
##  [8,] 0.15 0.40 0.45
##  [9,] 0.10 0.20 0.70
## [10,] 0.20 0.25 0.55
## [11,] 0.25 0.40 0.35
## [12,] 0.35 0.20 0.45
```

Basic structure check

Before testing the bootstrap procedure, I first check that the example has the basic structure needed for the AUC calculation

The number of true labels should match the number of rows in the probability matrix. I also check that the class names match the probability column names

I am only doing these checks here to make sure my development example is correctly structured. The complete package-level data validation belongs to `check_mc_data()`, so I do not want to repeat all of that validation inside `mc_auc_ci()`

```r
#check-structure
length(truth)
```

```
## [1] 12
```

```r
nrow(probabilities)
```

```
## [1] 12
```

```r
levels(truth)
```

```
## [1] "A" "B" "C"
```

```r
colnames(probabilities)
```

```
## [1] "A" "B" "C"
```

Original Multiclass AUC

Before doing any bootstrap resampling, I calculate the multiclass AUC using the original observations

I keep this as the original point estimate. The bootstrap procedure does not replace this value. Instead, I use repeated resampling to investigate the uncertainty around it

The underlying Hand-Till multiclass AUC calculation is performed using `pROC`. My function is responsible for the bootstrap procedure around this calculation

```r
#original-auc
original_roc <- pROC::multiclass.roc(response = truth, predictor = probabilities)
original_auc <- as.numeric(original_roc$auc)
original_auc
```

```
## [1] 0.9322917
```

The original AUC is still the point estimate from the observed data. The bootstrap AUCs are used to estimate its uncertainty rather than replacing the original estimate

Testing One Stratified Bootstrap Sample

Before running 1,000 bootstrap replicates, I first test one replicate so that I can see exactly what the resampling is doing

The important part here is that I resample within each class

For example, suppose a dataset contains:

- 80 observations from class A
- 15 observations from class B

- 5 observations from class C

If I resample all 100 observations together without considering the classes, a small class such as C may be poorly represented in a particular bootstrap sample

Instead, with the stratified approach I separately sample:

- 80 observations from the original A observations
- 15 from the original B observations
- 5 from the original C observations with replacement.

This means that every bootstrap replicate still contains all three classes and preserves the observed class counts.

```
#one-bootstrap
classes <- levels(truth)
set.seed(42)
boot_index <- unlist(
  lapply(classes, function(class_name) {
    class_index <- which(truth == class_name)
    sample(
      class_index,
      size = length(class_index),
      replace = TRUE)}),
  use.names = FALSE)
boot_index
```

```
##  [1]  1  1  1  1  6  8  6  6  9 12 11 12
```

`replace = TRUE` is important because this is bootstrap resampling

For example, if class A contains observations 1, 2, 3 and 4, one bootstrap sample might select:

`1, 1, 3, 4`

Observation 1 can appear more than once, while observation 2 may not appear in that replicate. However, all four sampled observations still come from class A

```
#check-bootstrap-classes
table(truth)
```

```
## truth
## A B C
## 4 4 4
```

```
table(truth[boot_index])
```

```
##
## A B C
## 4 4 4
```

Here I checked the stratification directly. Each class is resampled separately, so all classes remain in the bootstrap replicate

The sampled indices are applied to both the true labels and the probability matrix so that each label remains paired with its original predicted probabilities

I then calculate the multiclass AUC for this bootstrap sample

```
# first-bootstrap-auc
boot_truth <- truth[boot_index]
boot_probabilities <- probabilities[boot_index,,drop = FALSE]
boot_roc <- pROC::multiclass.roc(
  response = boot_truth,
  predictor = boot_probabilities)

boot_auc <- as.numeric(boot_roc$auc)
boot_auc
```

```
## [1] 0.9739583
```

I use exactly the same `boot_index` for both `truth` and `probabilities`.

This is important because the true label and predicted probabilities belong to the same observation and must stay together during resampling

For example, if observation 5 is selected into the bootstrap sample, I need both `truth[5]` and row 5 of the probability matrix. Resampling them separately could mismatch a true label with probabilities from a different observation and would make the AUC calculation incorrect

Repeating the Bootstrap

After checking that one stratified replicate works, I repeat the same procedure many times

For the prototype I use 1000 bootstrap replicates. I create a vector of 1000 `NA` values before starting the loop. Each successful bootstrap AUC will replace one of these `NA` values.

I chose this structure because it also gives me a simple way to keep track of failed replicates. If one AUC calculation fails, its position can remain `NA` rather than stopping the complete bootstrap procedure

```
# repeated-bootstrap
n_boot <- 1000
bootstrap_values <- rep(NA_real_, n_boot)
set.seed(42)

for (b in seq_len(n_boot)) {
  boot_index <- unlist(
    lapply(classes, function(class_name) {
      class_index <- which(truth == class_name)
      sample(
        class_index,
        size = length(class_index),
        replace = TRUE)}),use.names = FALSE)

  boot_truth <- truth[boot_index]
  boot_probabilities<-probabilities[boot_index,,drop=FALSE]
```

```
  bootstrap_values[b] <- tryCatch(
    {boot_roc <- pROC::multiclass.roc(response = boot_truth,predictor = boot_probabilities)

      as.numeric(boot_roc$auc)},
    error = function(e) {NA_real_})}
```

Handling a failed replicate

I use `tryCatch()` around the AUC calculation so that one failed bootstrap replicate does not stop all of the remaining replicates

For example, if replicate 420 fails but the previous 419 replicates were successful, I do not want to lose all of those results. I store `NA` for replicate 420 and continue to replicate 421

The failed replicate is not treated as an AUC value and is not used to calculate the standard error or confidence interval

However, I still count the failures. If many replicates fail, I would not simply ignore this. A high failure count could indicate that the data or the resampling procedure needs further investigation

```
#bootstrap-counts
successful_values <- bootstrap_values[!is.na(bootstrap_values)]
n_successful <- length(successful_values)
n_failed <- n_boot - n_successful
n_successful
```

```
## [1] 1000
```

```
n_failed
```

```
## [1] 0
```

For example, if I request 1,000 bootstrap replicates and 995 produce valid AUC values, the function would report:

- successful = 995
- failed = 5

Only the 995 successful AUC values would be used for the uncertainty calculations.

I keep the failure count in the output because it gives the user more information than silently removing failed replicates

Bootstrap Standard Error

I use the standard deviation of the successful bootstrap AUC estimates as the bootstrap estimate of standard error.

The idea is that the spread of the bootstrap AUC values tells me how much the estimate changes across resampled datasets.

For example, if the bootstrap values are very close:

`0.81, 0.82, 0.81, 0.82, 0.81`

the standard error should be relatively small.

If they vary more:

`0.70, 0.82, 0.76, 0.89, 0.73`

the standard error should be larger.

So I am using the variability across bootstrap AUC estimates to describe the uncertainty around the original AUC estimate.

```
#bootstrap-se
bootstrap_se <- sd(successful_values)
bootstrap_se
```

```
## [1] 0.05097834
```

Initial Percentile Confidence Interval

For the first working prototype, I use a percentile bootstrap confidence interval.

For a 95% confidence level:

$$\alpha = 1 - 0.95 = 0.05$$

The lower limit is therefore taken from:

$$\alpha/2 = 0.025$$

and the upper limit from:

$$1 - \alpha/2 = 0.975$$

So I use the 2.5th and 97.5th percentiles of the successful bootstrap AUC values.

For example, if most bootstrap AUC estimates fall roughly between 0.74 and 0.88, the percentile interval may be approximately:

`[0.74, 0.88]`

This is my initial CI method so that I can complete and test the bootstrap workflow. I am not assuming yet that the percentile interval is the best final method.

The next important step is to test its empirical coverage through simulation. If the coverage is not satisfactory, especially for small or imbalanced datasets, I will discuss whether another method such as bias-corrected and accelerated (BCa) should also be evaluated

```
#bootstrap-ci
conf_level <- 0.95
alpha <- 1 - conf_level
ci <- quantile(
  successful_values,
```

```
  probs = c(alpha / 2, 1 - alpha / 2),
  names = FALSE)
ci
```

```
## [1] 0.8071615 1.0000000
```

I used percentile CI to complete the first working prototype. My next step is to test its empirical coverage before deciding whether it should remain the final method

Retaining the Bootstrap Values

I do not remove the bootstrap AUC values after calculating the standard error and confidence interval

There are two reasons for keeping them

First, they are useful for checking the bootstrap distribution during development

Second, the visualisation part of the package can use the same values later. For example, a histogram of the bootstrap AUC estimates can be created without running the bootstrap procedure again

This also helps keep the responsibilities separate: `mc_auc_ci()` generates and stores the bootstrap results, while the visualisation component is responsible for displaying them

```
#inspect-bootstrap-values
head(successful_values)
```

```
## [1] 0.9739583 0.9375000 0.9375000 0.9322917 0.8906250 0.9843750
```

```
length(successful_values)
```

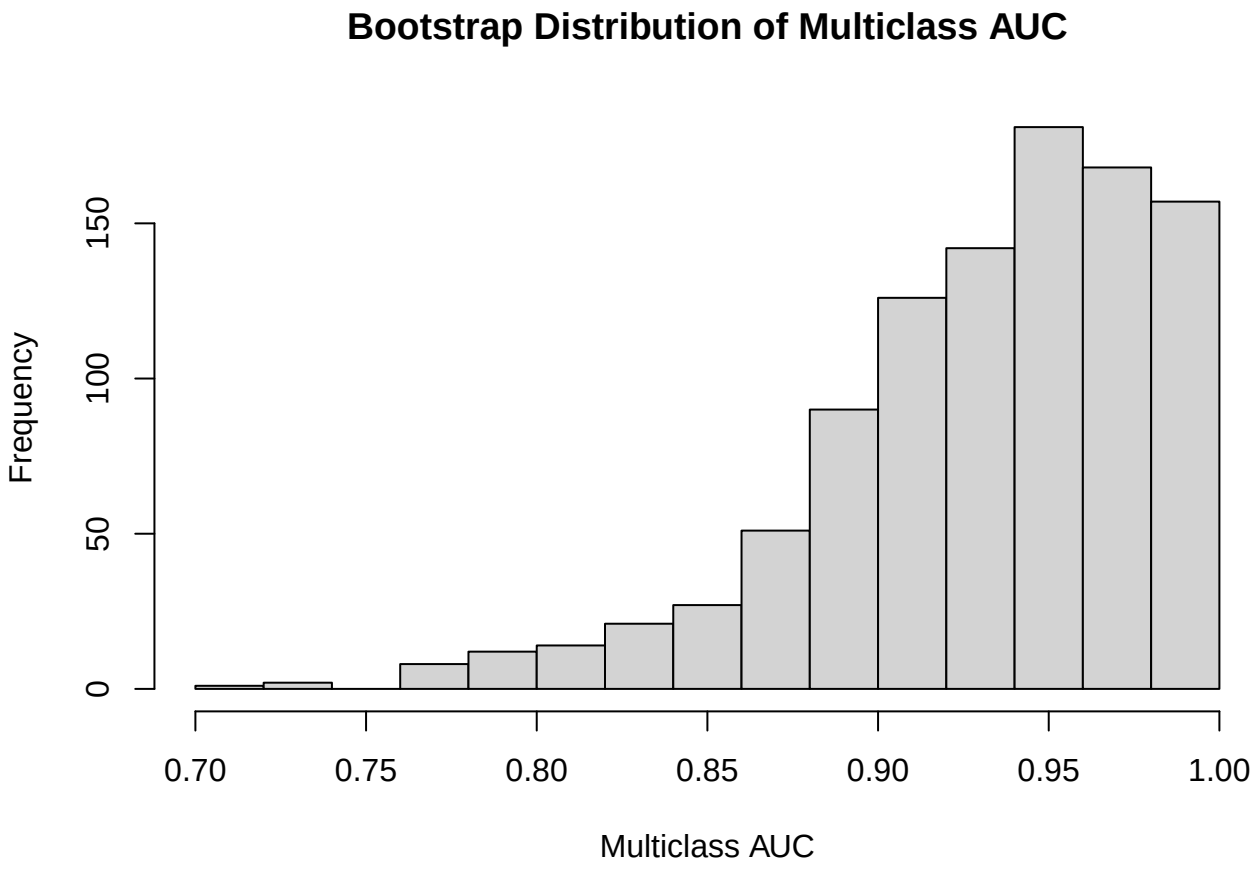
```
## [1] 1000
```

Inspecting the Bootstrap Distribution

The histogram below is only a development check. I use it to visually inspect whether the bootstrap AUC values actually vary and whether there are any obvious unusual results.

The histogram itself will not be part of the responsibility of `mc_auc_ci()`. The final package visualisation will be handled separately.

```
# development-histogram, fig.cap="Bootstrap distribution of multiclass AUC estimates
hist(
  successful_values,
  main = "Bootstrap Distribution of Multiclass AUC",
  xlab = "Multiclass AUC")
```



Just development plot.

Prototype Output

The main results are collected into one object. This makes the different outputs easy to access and also allows the bootstrap values to be passed to later visualisation components

```
# prototype-result
prototype_result <- list(
  auc = original_auc,
  ci_lower = ci[1],
  ci_upper = ci[2],
  se = bootstrap_se,
  successful = n_successful,
  failed = n_failed,
  bootstrap_values = successful_values,
  conf_level = conf_level,
  n_boot = n_boot)

prototype_result[
```

```
  c(
    "auc",
    "ci_lower",
    "ci_upper",
    "se",
    "successful",
    "failed")]
```

```
## $auc
## [1] 0.9322917
##
## $ci_lower
## [1] 0.8071615
##
## $ci_upper
## [1] 1
##
## $se
## [1] 0.05097834
##
## $successful
## [1] 1000
##
## $failed
## [1] 0
```

Initial `mc_auc_ci()` Function

After checking each part separately, I combine the procedure into an initial function.

At this stage, the function accepts the true labels and predicted probabilities directly. The final package version will need to use the common object created by `mc_roc()`. I will update the interface once the structure of that shared object is confirmed by the team

I include a small number of safeguards inside `mc_auc_ci()` for arguments that belong directly to this function.

For example:

- `n_boot = 0` does not make sense because no bootstrap replicates would be generated
- `n_boot = 10.5` does not make sense because the number of replicates needs to be a whole number
- `conf_level = 1.5` is not a valid confidence level.

These checks are different from the complete data-readiness checks.

I am not trying to duplicate the role of`check_mc_data()`.

```
mc_auc_ci <- function(truth,
                      probabilities,
                      n_boot = 1000,
                      conf_level = 0.95,
                      seed = NULL) {
```

```r
truth <- factor(truth)
probabilities <- as.matrix(probabilities)

# Basic safeguards needed for this function
if (length(truth) != nrow(probabilities)) {
  stop("truth and probabilities must have the same number of observations.")}

if (nlevels(truth) < 3) {
  stop("At least three classes are required.")}

if (!is.numeric(n_boot) ||
    length(n_boot) != 1 ||
    is.na(n_boot) ||
    n_boot < 1 ||
    n_boot != as.integer(n_boot)) {
  stop("n_boot must be a positive integer.")}

if (!is.numeric(conf_level) ||
    length(conf_level) != 1 ||
    is.na(conf_level) ||
    conf_level <= 0 ||
    conf_level >= 1) {
  stop("conf_level must be a single value between 0 and 1.")}

if (!is.null(seed)) {set.seed(seed)}

# Original multiclass AUC
original_roc <- pROC::multiclass.roc(
  response = truth,
  predictor = probabilities)
original_auc <- as.numeric(original_roc$auc)
classes <- levels(truth)
# Store one value for each bootstrap replicate
bootstrap_values <- rep(NA_real_, n_boot)
for (b in seq_len(n_boot)) {
  # Resample separately within each class
  boot_index <- unlist(
    lapply(classes, function(class_name) {
      class_index <- which(truth == class_name)
      sample(
        class_index,
        size = length(class_index),
        replace = TRUE)}),
    use.names = FALSE)

  # Keep labels and probabilities paired
  boot_truth <- truth[boot_index]
  boot_probabilities <- probabilities[boot_index,,drop = FALSE]

  # If one replicate fails, record NA and continue
  bootstrap_values[b] <- tryCatch({
      boot_roc <- pROC::multiclass.roc(
        response = boot_truth,
```

```
        predictor = boot_probabilities)

      as.numeric(boot_roc$auc)},
    error = function(e) {NA_real_})}

# Separate successful and failed replicates
successful_values <- bootstrap_values[
  !is.na(bootstrap_values)]

n_successful <- length(successful_values)
n_failed <- n_boot - n_successful

# Minimum safeguard for estimating variability
if (n_successful < 2) {
  stop("Too few successful bootstrap replicates.")}

# Bootstrap standard error
bootstrap_se <- sd(successful_values)

# Initial percentile confidence interval
alpha <- 1 - conf_level

ci <- quantile(
  successful_values,
  probs = c(alpha / 2, 1 - alpha / 2),
  names = FALSE)

result <- list(
  auc = original_auc,
  ci_lower = ci[1],
  ci_upper = ci[2],
  se = bootstrap_se,
  successful = n_successful,
  failed = n_failed,
  bootstrap_values = successful_values,
  conf_level = conf_level,
  n_boot = n_boot)

class(result) <- "mc_auc_ci"
return(result)}
```

Current interface limitation

For development, my prototype currently accepts `truth` and `probabilities` directly. This makes it easier for me to test the bootstrap procedure independently.

I understand that this is not intended to be the final package interface. In the package workflow, the final `mc_auc_ci()` should use the common object created by `mc_roc()`.

Once the structure of that object is confirmed by the team, I plan to adapt the function so that it takes the required labels, probabilities and original AUC information from that object rather than asking the user to provide the same information again.

```r
# run-function
result <- mc_auc_ci(
  truth = truth,
  probabilities = probabilities,
  n_boot = 1000,
  conf_level = 0.95,
  seed = 42)

result[
  c(
    "auc",
    "ci_lower",
    "ci_upper",
    "se",
    "successful",
    "failed")]
```

```
## $auc
## [1] 0.9322917
##
## $ci_lower
## [1] 0.8071615
##
## $ci_upper
## [1] 1
##
## $se
## [1] 0.05097834
##
## $successful
## [1] 1000
##
## $failed
## [1] 0
```

Basic Internal Checks

These are simple development checks rather than the full package test suite.

I check that:

- the original AUC is a single numeric value
- the lower confidence limit is not greater than the upper limit
- successful and failed replicates add up to the requested number of bootstrap replicates
- the number of stored bootstrap values matches the number of successful replicates
- the stored AUC values remain between 0 and 1

These checks do not prove that the statistical method is correct but they help me catch simple implementation mistakes before moving to the simulation study.

```r
# basic internal checks
stopifnot(
  is.numeric(result$auc),
  length(result$auc) == 1,
  result$ci_lower <= result$ci_upper,
  result$successful + result$failed == result$n_boot,
  length(result$bootstrap_values) == result$successful,
  all(result$bootstrap_values >= 0 &
      result$bootstrap_values <= 1))

cat("Basic internal checks passed.\n")
```

```
## Basic internal checks passed.
```

Reproducibility Check

Bootstrap resampling is random, so running the function twice would normally produce different bootstrap samples.

For development and testing, I include an optional `seed`. If I run the function twice with the same seed and the same input data, I expect the same sequence of bootstrap values

The seed does not change the purpose of the statistical method. I use it mainly so that I can reproduce results when debugging, testing or showing the prototype

```r
# reproducibility-check
result_1 <- mc_auc_ci(truth,
                      probabilities,
                      n_boot = 100,
                      seed = 42)

result_2 <- mc_auc_ci(truth,
                      probabilities,
                      n_boot = 100,
                      seed = 42)

identical(
  result_1$bootstrap_values,
  result_2$bootstrap_values)
```

```
## [1] TRUE
```

Simple Argument Check

I also test one invalid function argument to make sure that the function stops with a clear message rather than continuing with an invalid bootstrap setting.

For example, `n_boot = 0` means that no bootstrap samples would be generated, so this should not be accepted.

```r
mc_auc_ci(
  truth = truth,
  probabilities = probabilities,
  n_boot = 0)
```

```
## Error in mc_auc_ci(truth = truth, probabilities = probabilities, n_boot = 0): n_boot must be a positi
```

This error is intentional. I'm testing the defensive behaviour of the function. If a user requests zero bootstrap replicates, the function should stop with a clear message rather than continue with an invalid setting

Current Status and Next Step

At this stage, I have a working first prototype of `mc_auc_ci()`.

So far, I have checked that the prototype can:

- calculate the original multiclass AUC using `pROC`
- resample observations separately within each class
- preserve all classes in each bootstrap replicate
- recalculate the multiclass AUC across repeated bootstrap samples
- continue when an individual bootstrap replicate fails
- report successful and failed replicates
- estimate the bootstrap standard error
- calculate an initial percentile confidence interval
- retain the bootstrap AUC values for later visualisation.

However, I do not consider the confidence interval method fully validated yet.

The current small example mainly tells me that the implementation is working in the way I intended. It does not tell me whether a nominal 95% confidence interval actually contains the target multiclass AUC approximately 95% of the time.

My next step is therefore simulation validation.

For a simulation scenario, I plan to generate data where the target multiclass AUC is known or can be approximated accurately. I will then:

1. generate one simulated dataset
2. run `mc_auc_ci()` to obtain a 95% confidence interval
3. check whether that interval contains the target AUC
4. repeat the process across many independently simulated datasets
5. calculate the proportion of intervals containing the target value.

For example, if I run 500 simulated datasets and 472 of the 500 intervals contain the target AUC, the estimated coverage would be:

$$472/500 = 0.944$$

or 94.4%.

I then want to compare this behaviour under different sample sizes, numbers of classes and levels of class imbalance, as suggested by the supervisor

Before I finalise this part, I would like to confirm with the supervisor:

1. whether my current stratified bootstrap implementation is following the intended direction
2. whether percentile CI is suitable as the first method to evaluate in the simulation
3. whether BCa or another interval method should also be included in the comparison
4. whether the proposed simulation scenarios are sufficient before I finalise the validation